

Graph Transformers

CPSC483: Deep Learning on Graph-Structured Data

Rex Ying

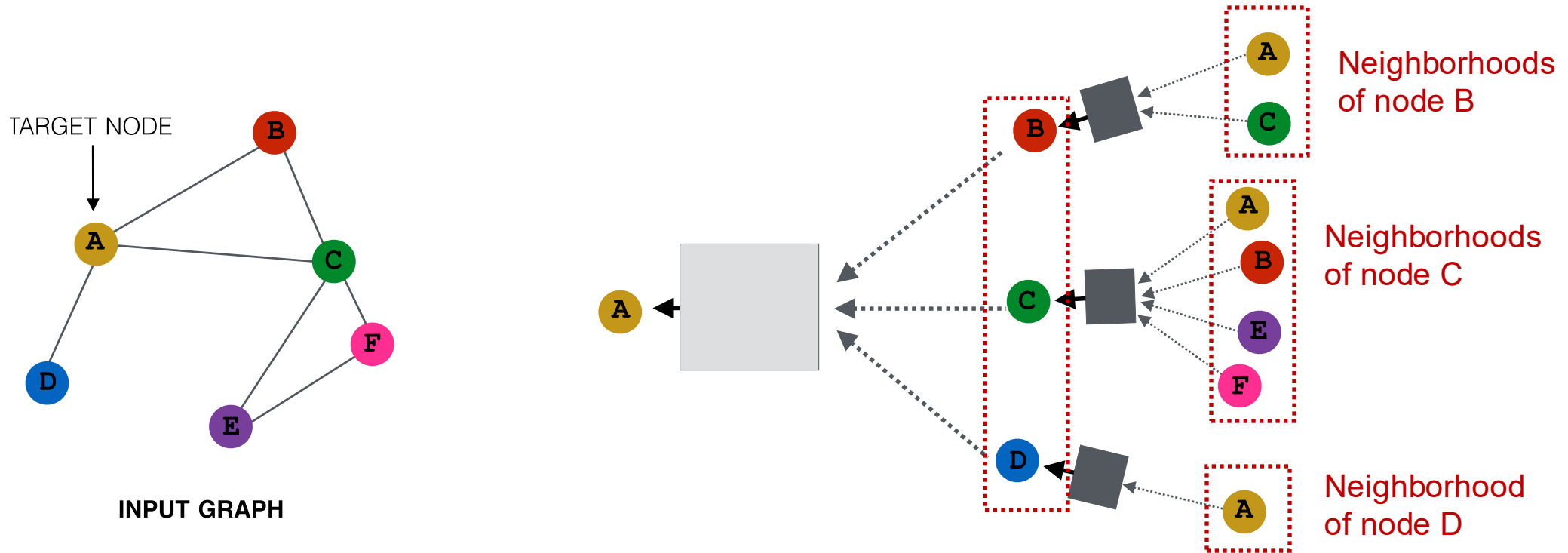
Content

- **Motivation of Graph Transformers**
- **Challenges for Building Graph Transformers**
- **Encodings: Positional & Structural**
 - Laplacian Positional Encoding
 - Random Walk Structural Encoding
- **Token Construction**
- **Forward Propagation**
- **Scalability**
- **Empirical Verification**

Content

- **Motivation of Graph Transformers**
- Challenges for Building Graph Transformers
- Encodings: Positional & Structural
 - Laplacian Positional Encoding
 - Random Walk Structural Encoding
- Token Construction
- Forward Propagation
- Scalability
- Empirical Verification

Recap: Graph Neural Networks



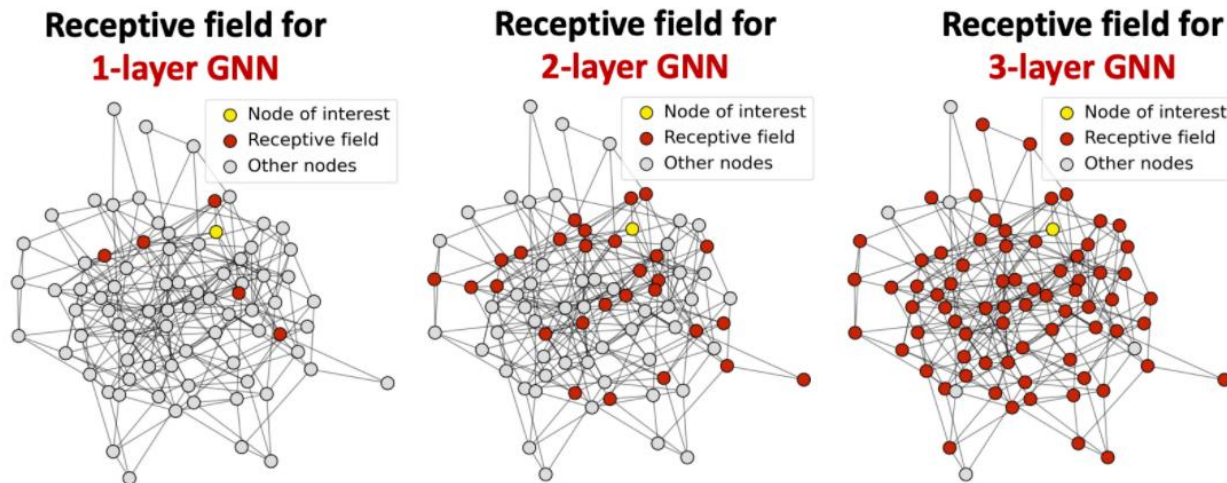
A single message-passing layer can only explore the relationship between node and its **one-hop** neighbors

Message Passing Drawbacks (1)

- **Expressiveness:** 1-order Message passing GNNs (**MPNN**) have limited expressiveness (at most as powerful as 1-WL test)
- **Over-smoothing:** node features tend to converge to the same value with the increasing number of message passing layers



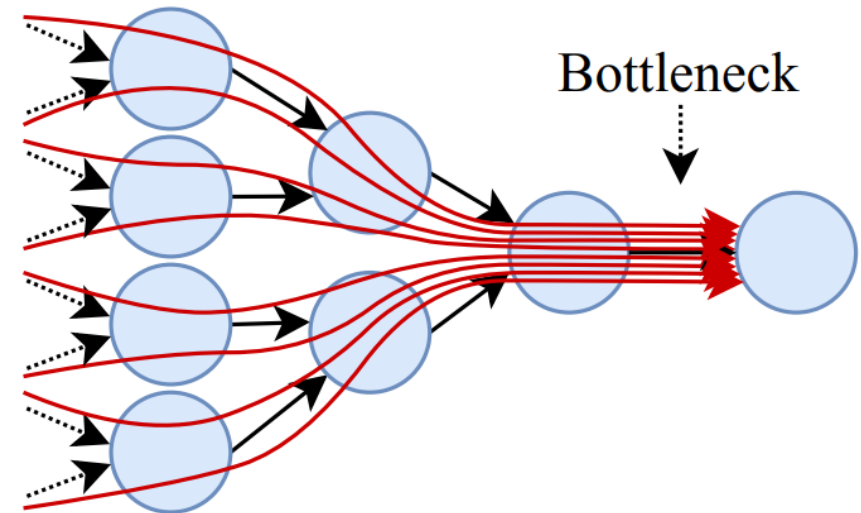
Non-isomorphic graphs that cannot be distinguished by MPNN



Receptive field quickly covers the entire graph as the number of layers increases

Message Passing Drawbacks (2)

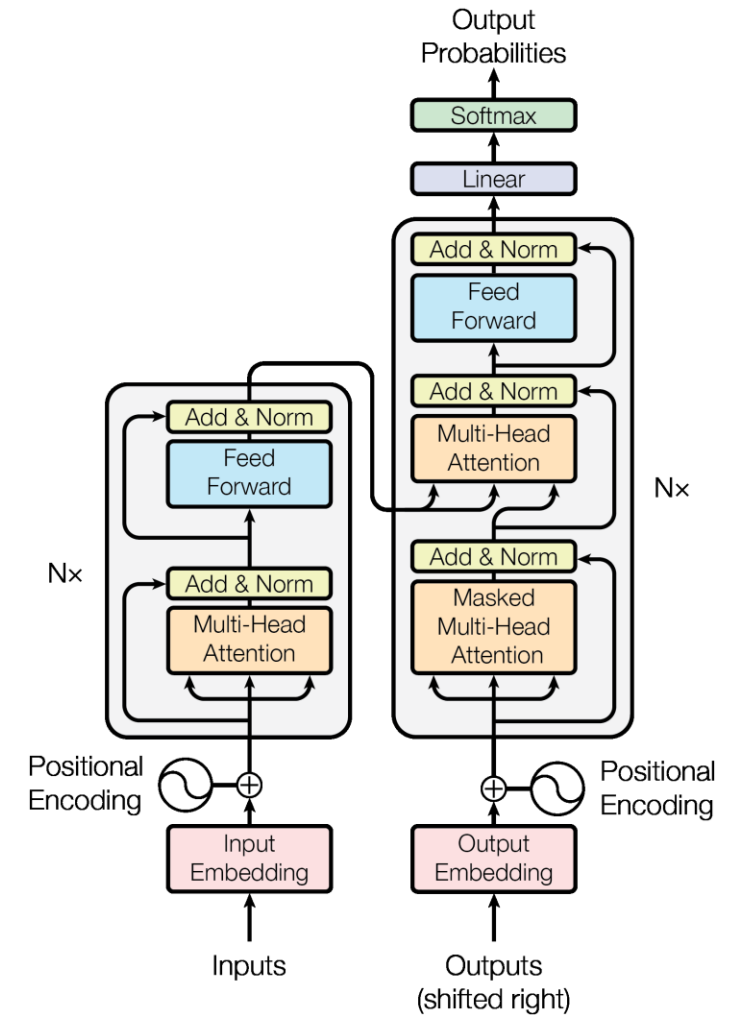
- **Over-squashing:** In tasks relying on long-distance interactions, an **exponentially-growing** amount of information from distant nodes is squashed into a fixed-size vector.
- Message Passing Layers cannot **capture long-range dependencies effectively.**



The bottleneck of graph neural networks

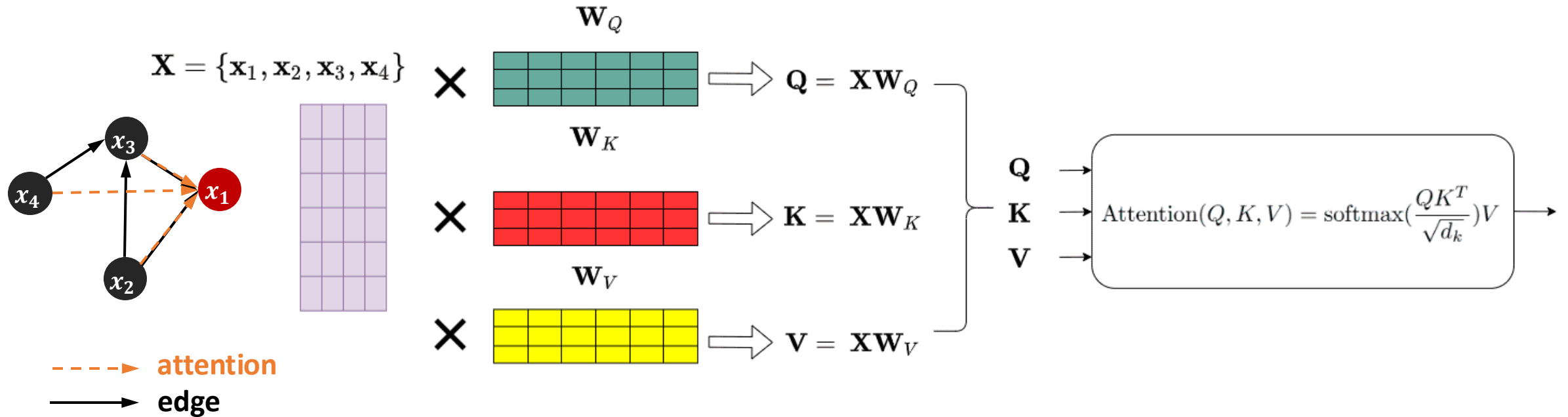
Motivation of Graph Transformer

- **Transformer is powerful in modelling sequential data**, such as natural language, speech, computer vision.
- Transformer **enables long-range connections** as all tokens attend to each other
- **Motivation**: Can Transformer architectures be used to model graphs and **improve graph representation learning**?
 - Expressiveness
 - Over-smoothing
 - Over-squashing (long-range)



Attention in Graph Transformers

Naïve full attention on node set:



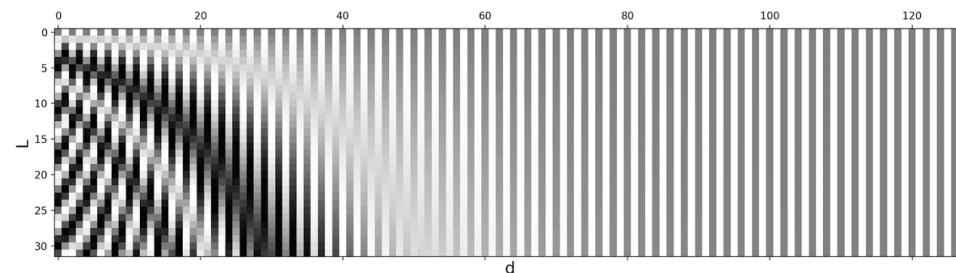
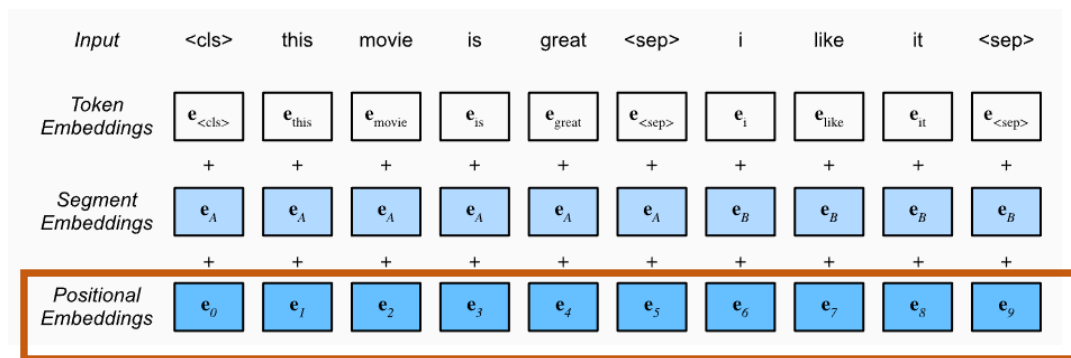
- Naturally capture **long-range dependencies**
- **Edge features** are not captured effectively

Content

- Motivation of Graph Transformers
- **Challenges for Building Graph Transformers**
- Encodings: Positional & Structural
 - Laplacian Positional Encoding
 - Random Walk Structural Encoding
- Token Construction
- Forward Propagation
- Scalability
- Empirical Verification

Challenges for Graph Transformers (1)

- Language has natural sequential order, while graphs are permutation invariant to node ordering.
- Recall Positional Encoding in Lecture 8:

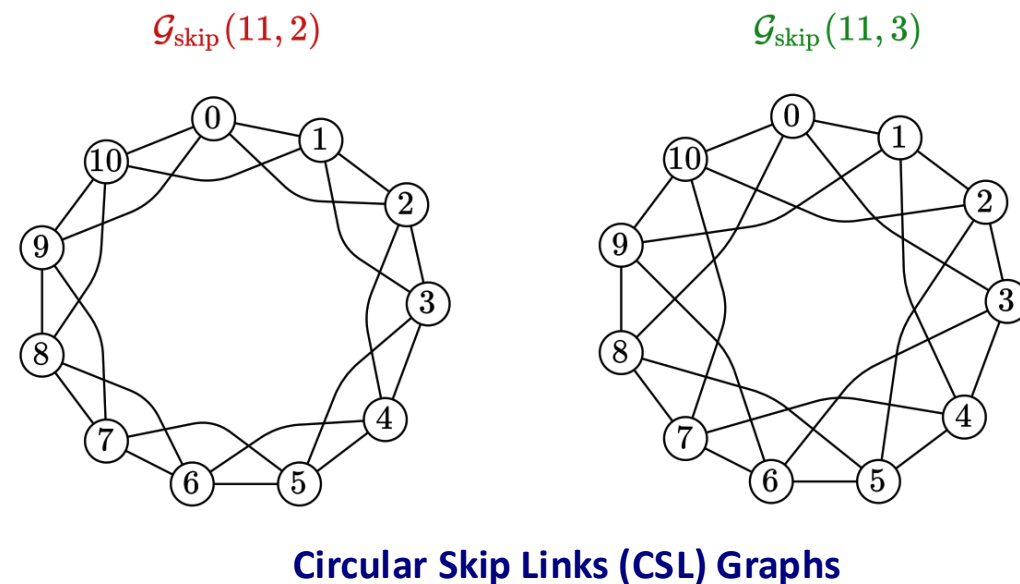


$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

- Positional Encoding for sequential input fails to identify nodes in a graph
- How to better **indicate the position of the node in a graph** $\Rightarrow$ **Positional Encoding (PE)**

Challenges for Graph Transformers (2)

- Message passing GNNs suffer from **limited expressiveness** (WL test)
- Message passing GNNs cannot capture **local structure information** sufficiently.
- How to **better incorporate neighborhood information** in graph transformers $\Rightarrow$ **Structural Encoding (SE)**



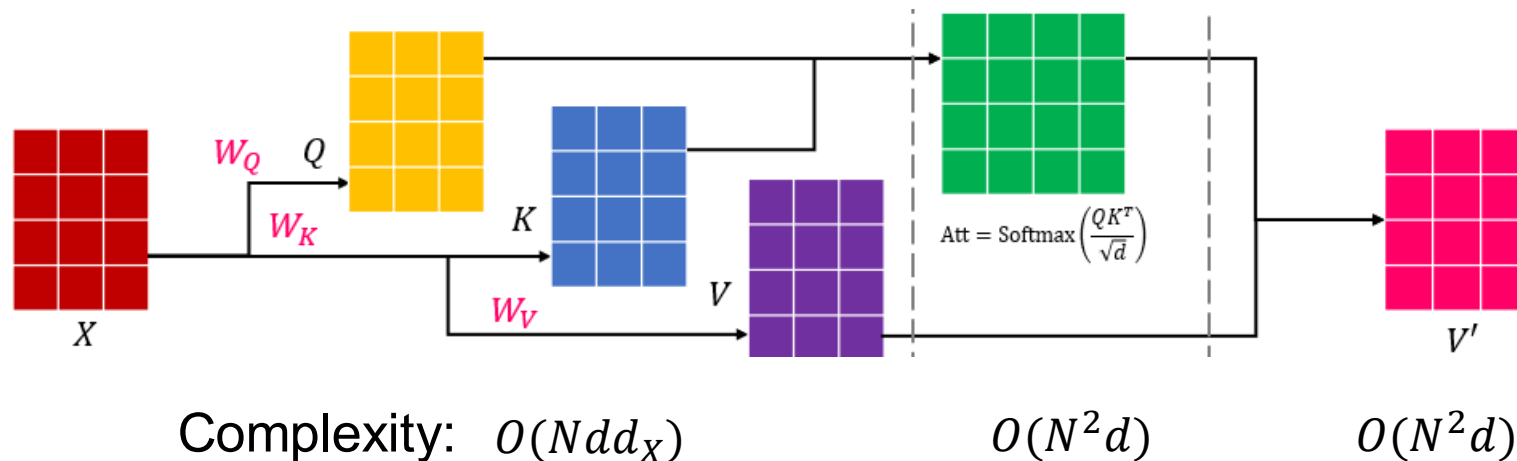
- Message passing GNNs **fail to distinguish** **Circular Skip Links (CSL)** graph pairs [1]
- **Structural Encoding** (which captures local substructure) can distinguish graph pairs[2]

[1] Murphy, Ryan, et al. "Relational pooling for graph representations." *International Conference on Machine Learning*. PMLR, 2019.

[2] Rampášek, Ladislav, et al. "Recipe for a general, powerful, scalable graph transformer." *Advances in Neural Information Processing Systems* 35 (2022): 14501-14515.

Challenges for Graph Transformers (3)

- Computational complexity of MPNN: $O(E)$, where E is the number of edges
- Conventional Transformers induce square computational complexity $O(N^2)$ in the number of nodes $N \Rightarrow$ **improve the scalability of graph transformers**
 - The QK^T matrix multiplication, Softmax(), AttV value updates **in full attention** all consume n^2 time and memory.

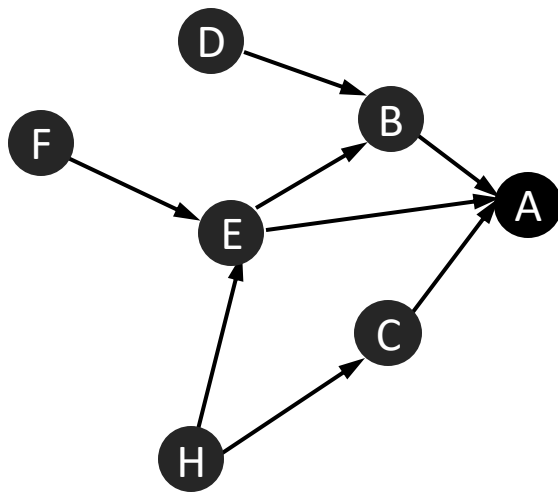


Content

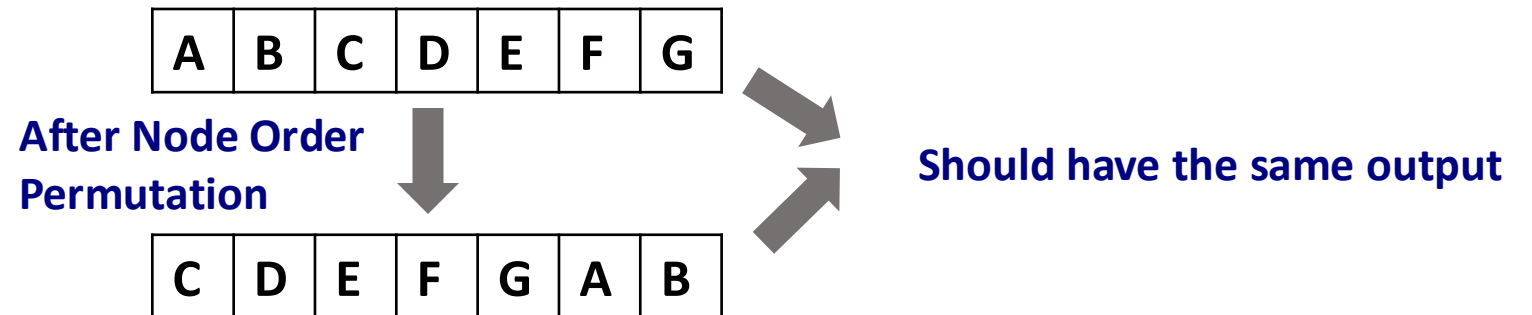
- Motivation of Graph Transformers
- Challenges for Building Graph Transformers
- **Encodings: Positional & Structural**
 - Laplacian Positional Encoding
 - Random Walk Structural Encoding
- Token Construction
- Forward Propagation
- Scalability
- Empirical Verification

Why traditional PE fails?

- Graphs **do not have a natural node ordering** like sequences.
- **Permutation equivariance** should be preserved by positional encoding.



No natural node ordering within a graph



Differences between PE and SE

Positional Encodings (PE)

- Provide **an embedding of the position of a given node within the graph**
- **Assumption:** when two nodes are **close** to each other within a graph or subgraph, their PE should also be close.

Structural Encodings (SE)

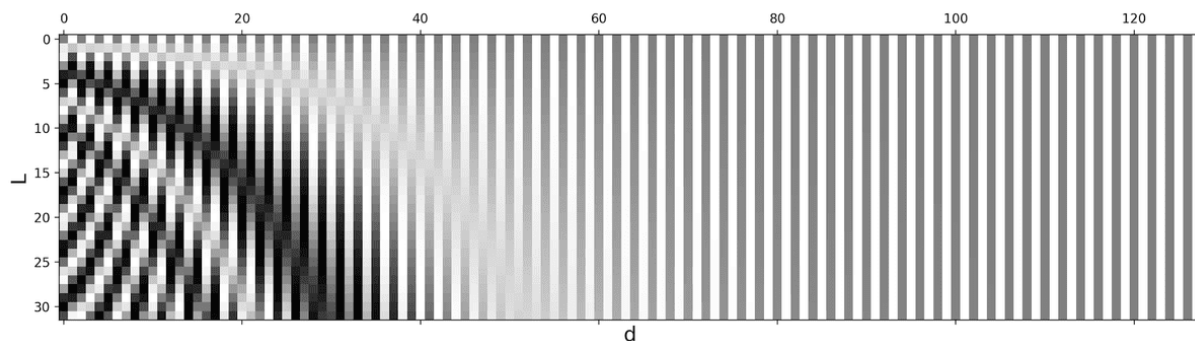
- Provide **an embedding of the structure of neighborhoods or subgraphs** to help increase the expressiveness and the generalizability of GNNs
- **Assumption:** when two nodes **share similar subgraphs**, or when two graphs are similar, their SE should also be close.

Content

- Motivation of Graph Transformers
- Challenges for Building Graph Transformers
- Encodings: Positional & Structural
 - **Laplacian Positional Encoding**
 - Random Walk Structural Encoding
- Token Construction
- Forward Propagation
- Scalability
- Empirical Verification

Why Sin/Cos as Positional Encoding (1)

- Recall the positional encoding for sequential input:



$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

- Question:** Why we use Sine/Cosine functions as positional encoding?
- In Euclidean space, **sine/cosine functions are the eigenfunctions of the Laplacian operator** f , i.e., $Lf = \lambda f$ with some λ .

Definition of Laplacian Operator:

$$Lf = \text{div}(\nabla f)$$

Why Sin/Cos as Positional Encoding (2)

- **Example:** in $\mathbb{R}^2$, $\sin(mx + ny)$, $\cos(mx + ny)$ are eigenfunctions of L , with $\lambda = -(m^2 + n^2)$.
- **Theorem [Fourier Series]:** any square-integrable function $F(x, y)$ on $\mathbb{R}^2$, i.e.,

$$\iint_{-\infty}^{\infty} |F(x, y)| \, dx \, dy < \infty$$

can be expanded into

$$F(x, y) = \sum_{m, n=-\infty}^{+\infty} a_{m, n} \sin(mx + ny) + b_{m, n} \cos(mx + ny)$$

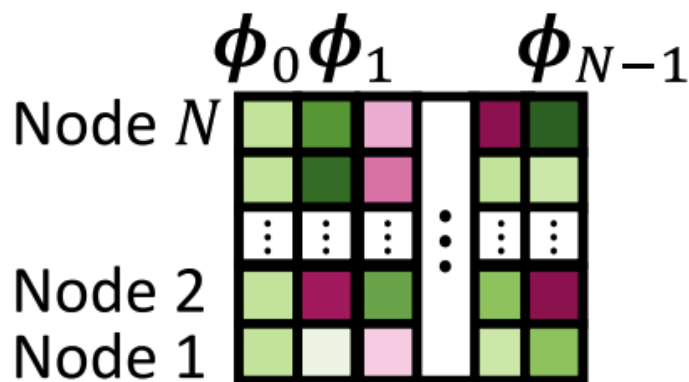
where $a_{m, n}$ and $b_{m, n}$ are Fourier coefficients

Laplacian in Graph Domain

- In graph domain, the eigenvectors of the graph Laplacian naturally encode the positional information of the given graph

$$L = I - D^{-\frac{1}{2}} A D^{-\frac{1}{2}} = U^T \Lambda U$$

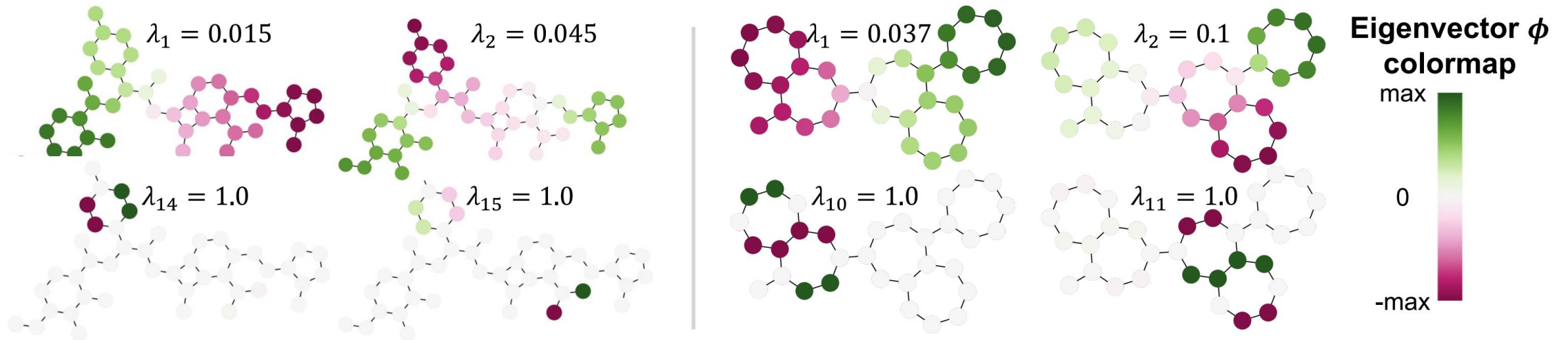
- $U = [\phi_0, \dots, \phi_{N-1}]^T$, where ϕ_i indicates the i -th eigenvector



- Each column represents an eigenvector
- One row per node.

Eigenvectors Reflect Graph Substructures

- Eigenvectors can be used to discriminate between different graph structures and sub-structures



Kreuzer, Devin, et al. "Rethinking graph transformers with spectral attention." *Advances in Neural Information Processing Systems* 34 (2021): 21618-21629.

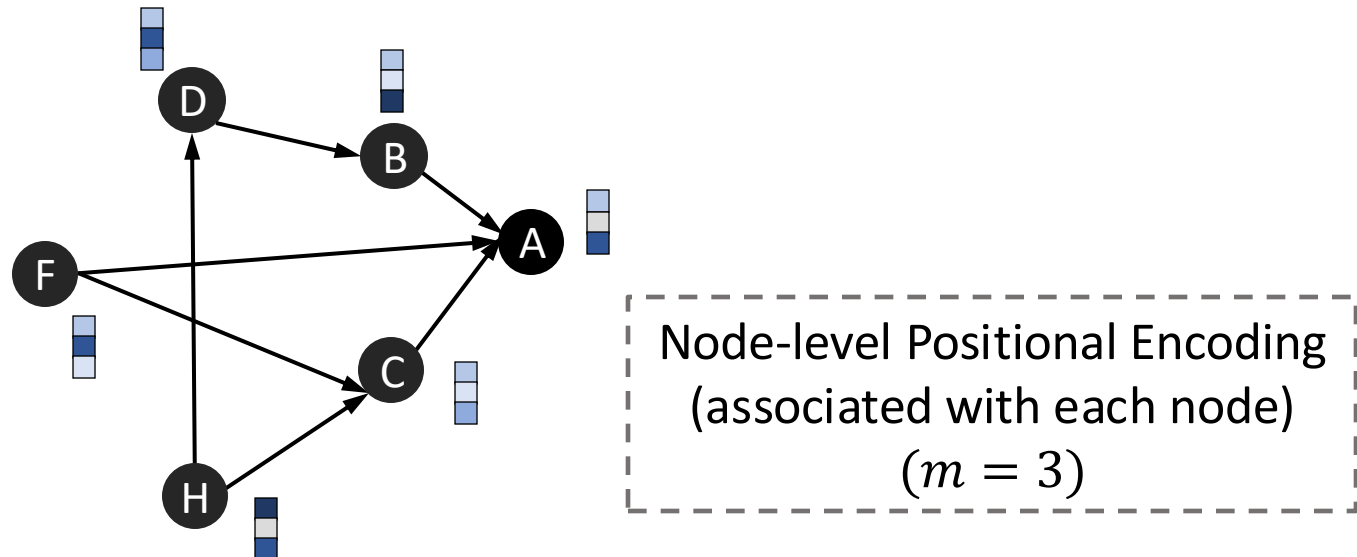
The **low-frequency** eigenvectors ϕ_1, ϕ_2 are **spread across the graph**

The **high-frequency** eigenvectors ϕ_i ($i > 10$) **resonate in local structures**

Laplacian Positional Encoding (1)

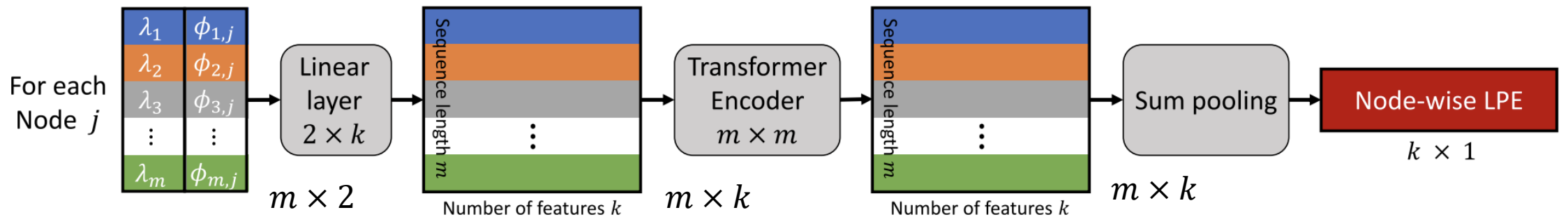
- Use Laplacian eigenvectors as node positional encoding (usually select m eigenvectors corresponding to **m -lowest non-zero eigenvalues**)
- The Laplacian positional Encoding **for the j -th node**:

$$p_j^{\text{LapPE}} = [\phi_{1j}, \phi_{2j}, \dots, \phi_{mj}]$$



Laplacian Positional Encoding (2)

- Learnable positional encoding
 - concatenate with the corresponding eigenvalues
 - learn the potential positional encoding by neural networks



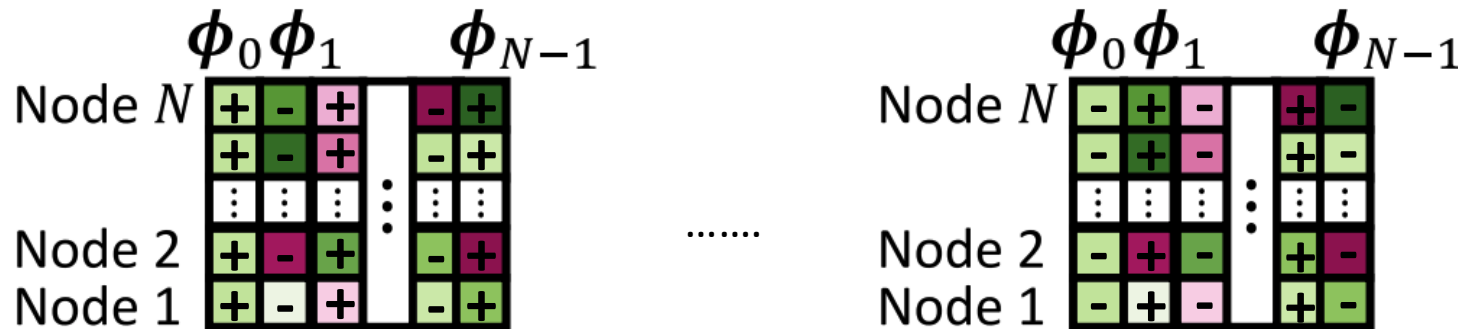
LPE: Learned Positional Encoding

[1] Kreuzer, Devin, et al. "Rethinking graph transformers with spectral attention." *Advances in Neural Information Processing Systems* 34 (2021): 21618-21629.

Sign Ambiguity in LapPE

- If ϕ_i is an eigenvector, then $-\phi_i$ is also an eigenvector.
- There is a total of 2^m possible sign combinations when choosing m **eigenvectors of a graph**

- Example:



8 possible 3-dim Laplacian Positional Encodings in total

Solutions to Sign Ambiguity

- **Randomly flip the sign** of eigenvectors during training to encourage model invariance to with respect to signs [1]
- Apply **permutation equivariant mapping** [2] :

$$\text{PE} = [f(\phi_1) + f(-\phi_1), \dots, f(\phi_m) + f(-\phi_m)]$$

- $f: \mathbb{R}^N \rightarrow \mathbb{R}^N$ can be any permutation equivariant networks, such as [DeepSets](#) , Transformers or GNNs
- Node Embedding Matrix $\text{PE} \in \mathbb{R}^{N \times m}$
- N is the number of nodes

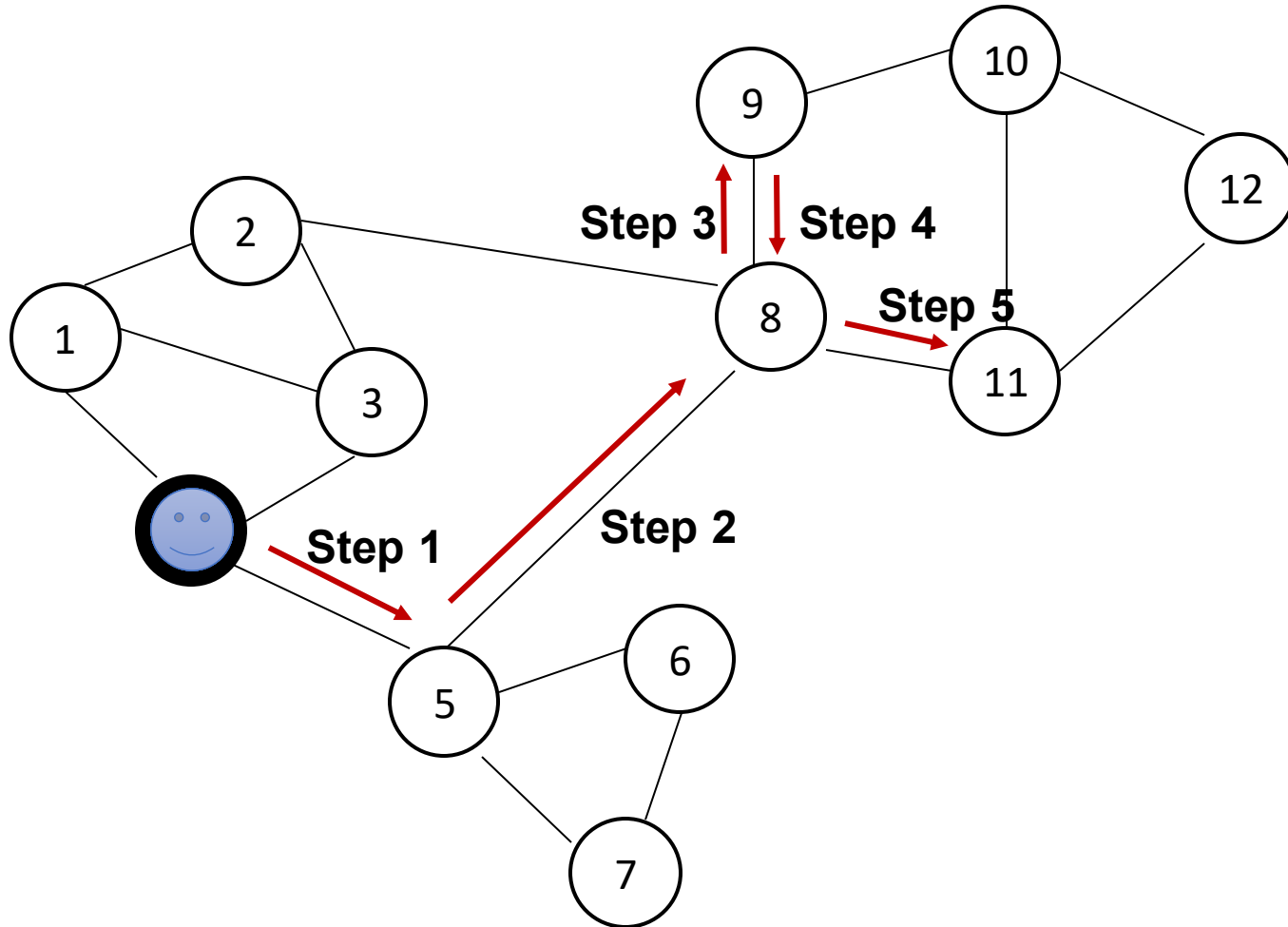
[1] Dwivedi, Vijay Prakash, et al. "Benchmarking graph neural networks." *arXiv preprint arXiv:2003.00982* (2020).

[2] Lim, Derek, et al. "Sign and basis invariant networks for spectral graph representation learning." *arXiv preprint arXiv:2202.13013* (2022).

Content

- Motivation of Graph Transformers
- Challenges for Building Graph Transformers
- Encodings: Positional & Structural
 - Laplacian Positional Encoding
 - **Random Walk Structural Encoding**
- Token Construction
- Forward Propagation
- Scalability
- Empirical Verification

Random Walk



- Given a *graph* and a *starting point*, we **select a neighbor** of it at **random**
- Move to this neighbor
- Select a neighbor of this point at random, and move to it.
- The (random) sequence of points visited this way is a **random walk on the graph.**

Random Walk Encoding

- $RW = D^{-1}A$ is the **random walk operator**. A is the adjacency matrix, D is the degree matrix.
- **Random Walk Encoding (RWE)** for the i -th node is defined with k -steps of random walk as

$$p_i^{RWE} = [RW_{ii}, RW_{ii}^2, \dots, RW_{ii}^k] \in \mathbb{R}^k$$

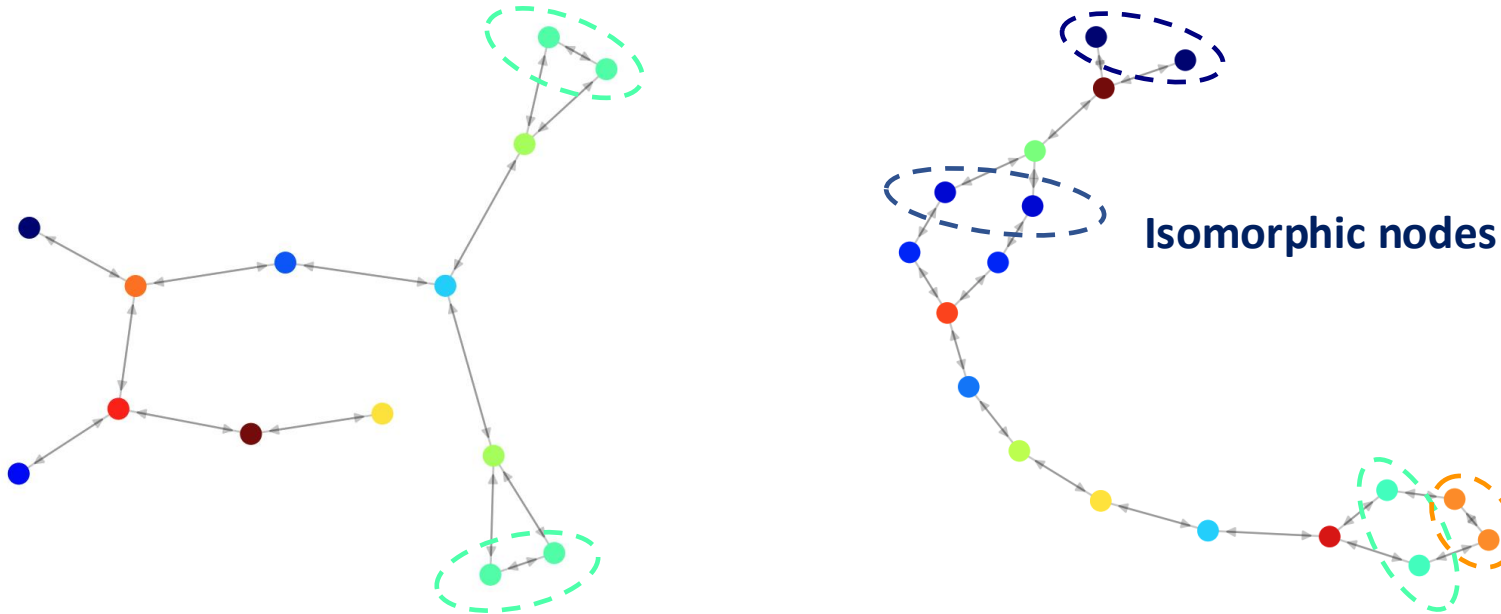
- RWE encodes the **landing probability** of a node to itself in 1 to k steps of random walk $\Rightarrow$ meaningful **higher-order** structure information!
- Note: RWE is a Structural Encoding.

Question:

1. What happens when k increases? (Higher-order neighborhood is considered)
2. What happens to the RWE if a node is densely connected to its neighborhood?

Dwivedi, Vijay Prakash, et al. "Graph neural networks with learnable structural and positional representations."

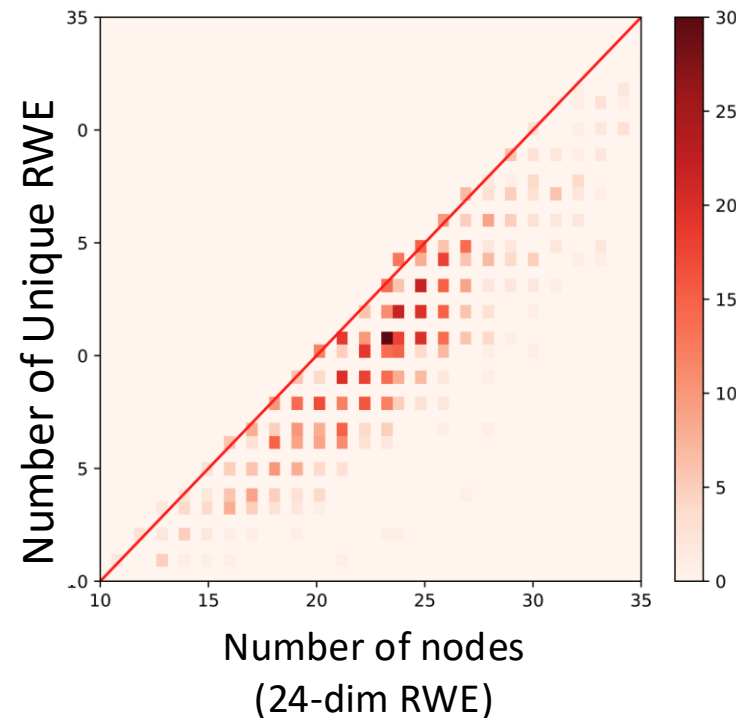
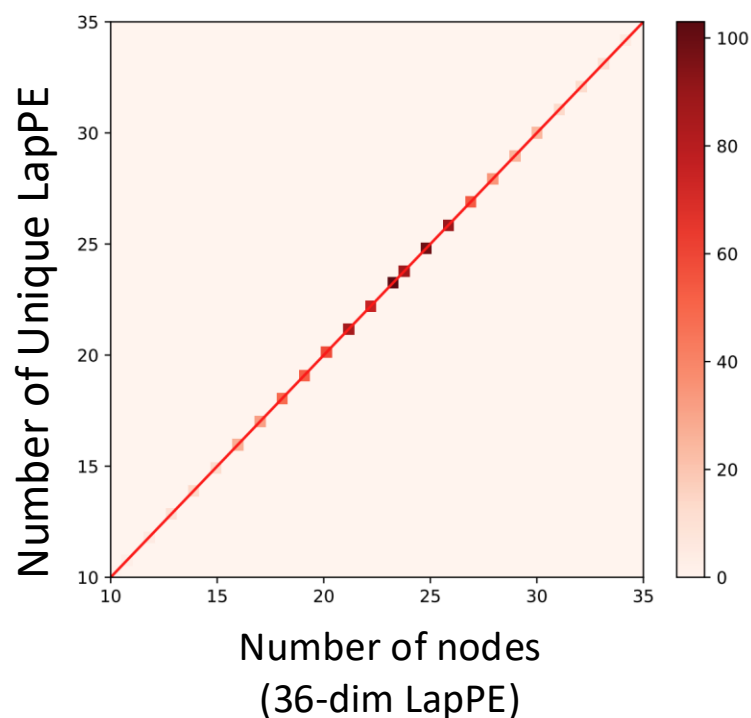
Random Walk Encoding Visualization



- **Different node color** represents a **unique RWE** vector with $k = 24$
- The nodes with the same color / RWE are isomorphic in the graph, i.e. their k -hop structural neighborhoods are the same!

Comparison between RW and Laplacian

LapPE and RWE on ZINC validation set (every square is a graph):

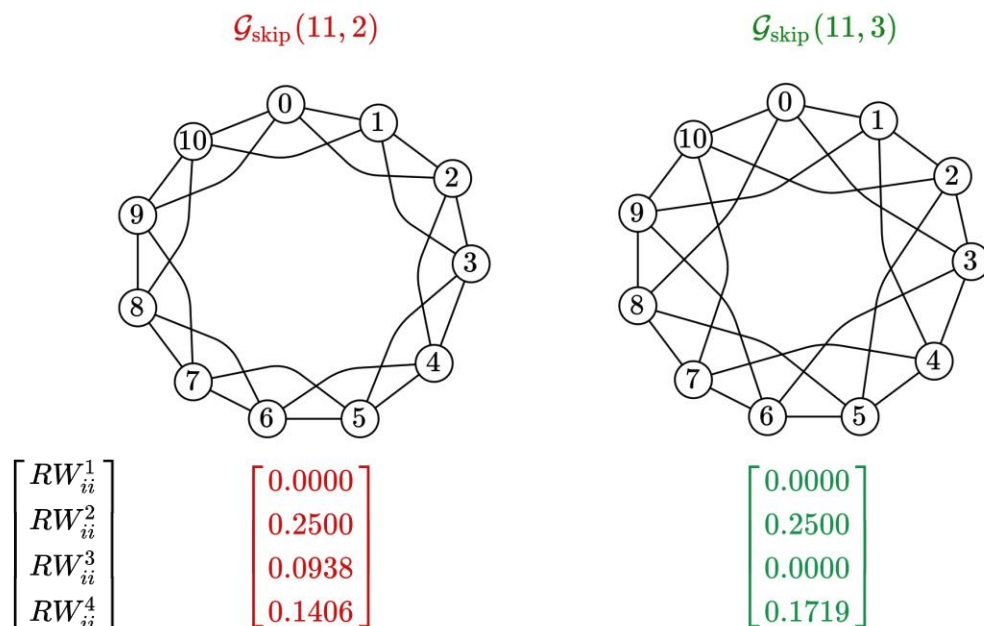


Color darkness indicates the number of graphs

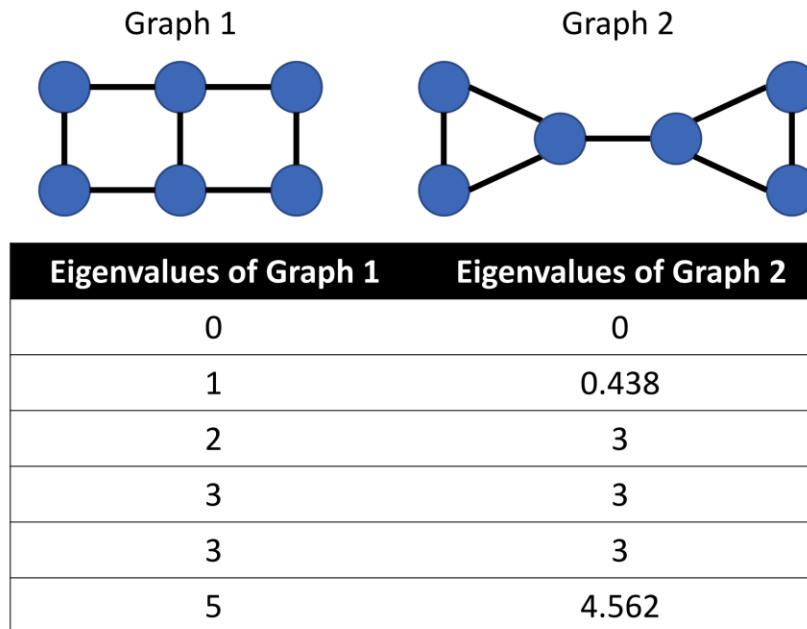
- Laplacian PE guarantees unique node representations better (an injective mapping)
- With RWE, most graphs have a large amount of nodes with unique positional encoding
- Note: RWE do not suffer from the sign ambiguity of LapPE $\Rightarrow$ more efficient training

PE Improves Expressiveness

- Positional Encoding can distinguish graph pairs that cannot be correctly distinguished by Message-passing GNNs (1-WL algorithm)



4-dimensional RWE distinguish CSL graph pairs

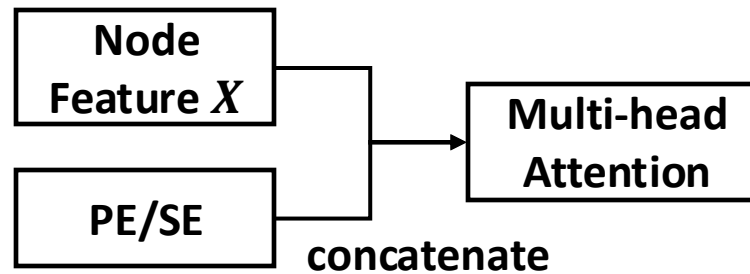


non-isomorphic graphs that can be distinguished by their eigenvalues but not by Message Passing GNNs

How to inject PE/SE into (1)

- **Concatenate with node/edge/graph features** before Attention layers

- Example: [SAN](#), [GPS](#)



- Inject PE/SE with **attention score**

- Example: [GraphiT](#)

$$\text{PosAttention}(Q, V, K_r) = \text{normalize} \left(\exp \left(\frac{Q Q^\top}{\sqrt{d_{\text{out}}}} \right) \odot \boxed{K_r} \right) V \in \mathbb{R}^{n \times d_{\text{out}}}$$

Graph kernel: where $K_r(i, j)$ encodes relative position between nodes i and j

How to inject PE/SE (2)

- Inject local PE/SE with **node inputs** and treat relative PE/SE as additional **attention bias**

- Example: [Graphormer](#)

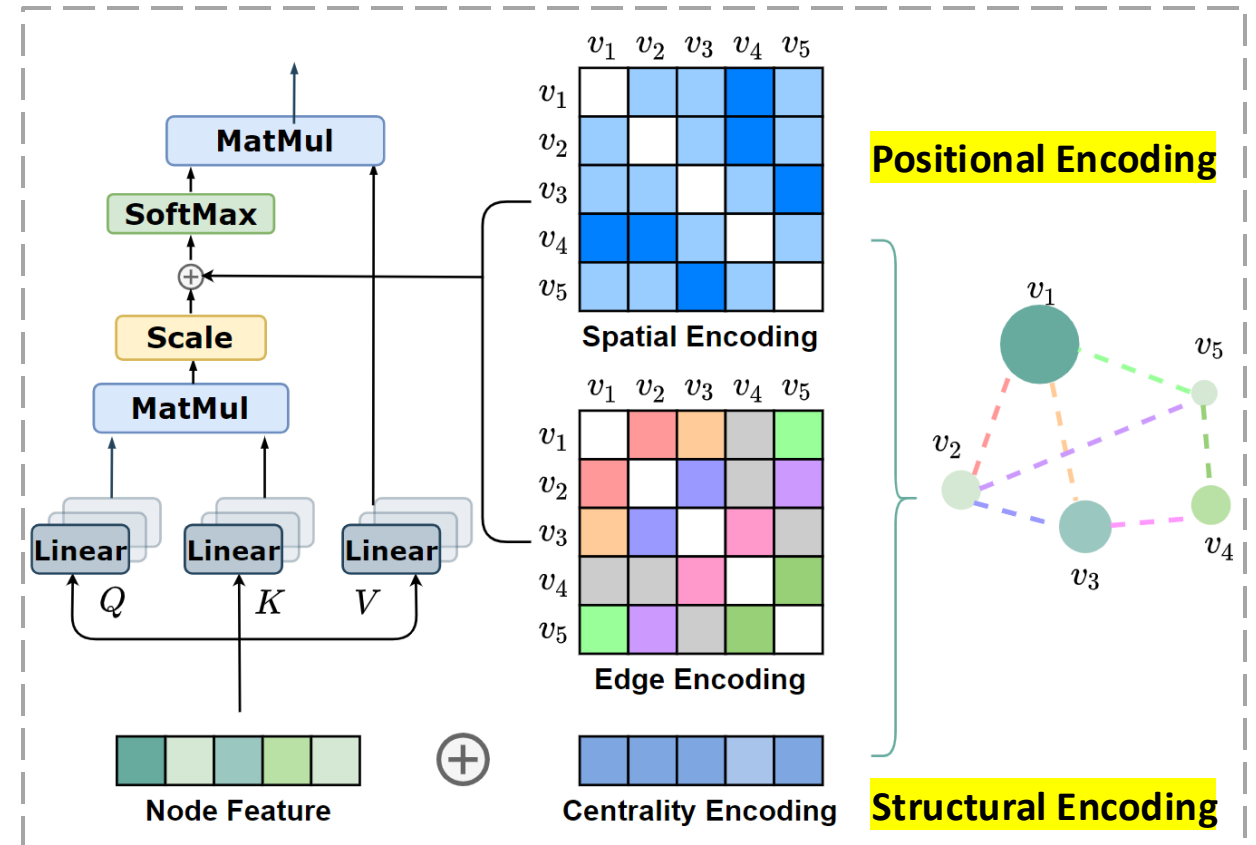
$$\text{Attention: } e_{ij} = \frac{(\mathbf{h}_i \mathbf{W}_q)(\mathbf{h}_j \mathbf{W}_k)^T}{\sqrt{d}} + b_{\phi(v_i, v_j)} + c_{ij}$$

Spatial Encoding:

Shortest path between v_i, v_j

Edge Encoding:

Average all edge features along the shortest path between v_i, v_j $c_{ij} = \frac{1}{N} \sum_{e \in SP(i,j)} x_e w_e$

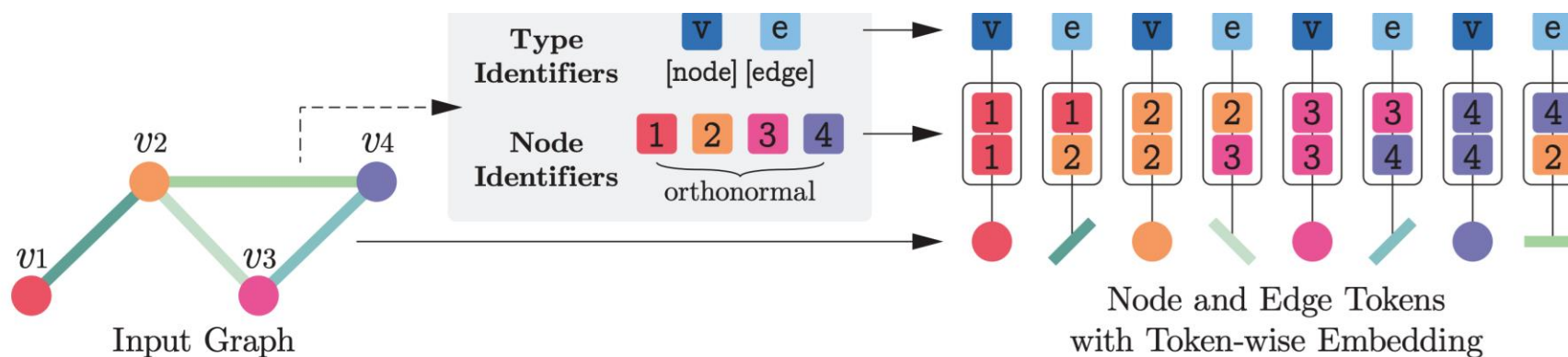


Content

- Motivation of Graph Transformers
- Challenges for Building Graph Transformers
- Encodings: Positional & Structural
 - Laplacian Positional Encoding
 - Random Walk Structural Encoding
- **Token Construction**
- Forward Propagation
- Scalability
- Empirical Verification

Token Construction (1)

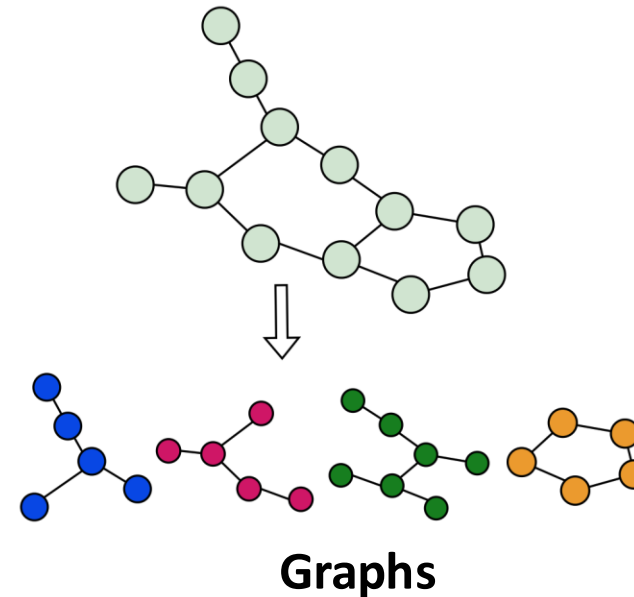
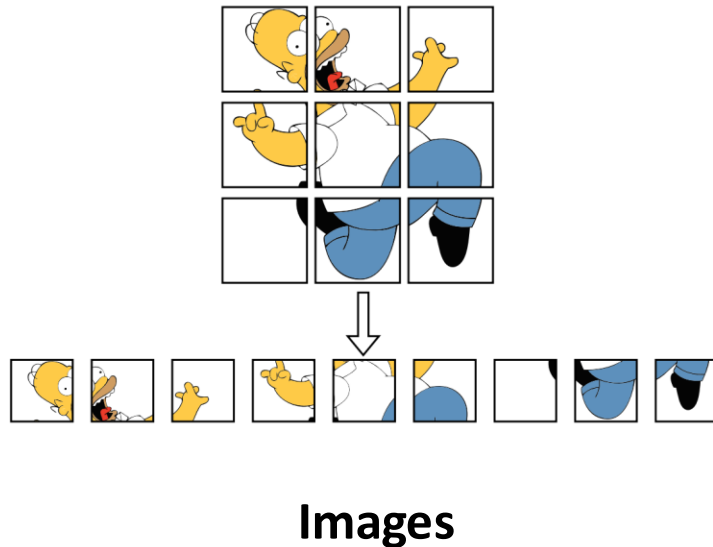
- Using nodes as input tokens is the most common approach
 - The complexity is $O(N^2)$ (graph transformers with full attention)
- Use nodes and edges as input tokens (Examples: [EGT](#), [TokenGT](#))
 - Model higher-order node-edge and edge-edge interactions $\Rightarrow$ **stronger expressiveness**
 - The complexity is $O(N + E)^2$



Kim, Jinwoo, et al. "Pure transformers are powerful graph learners." *Advances in Neural Information Processing Systems* 35 (2022): 14582-14595.

Token Construction (2)

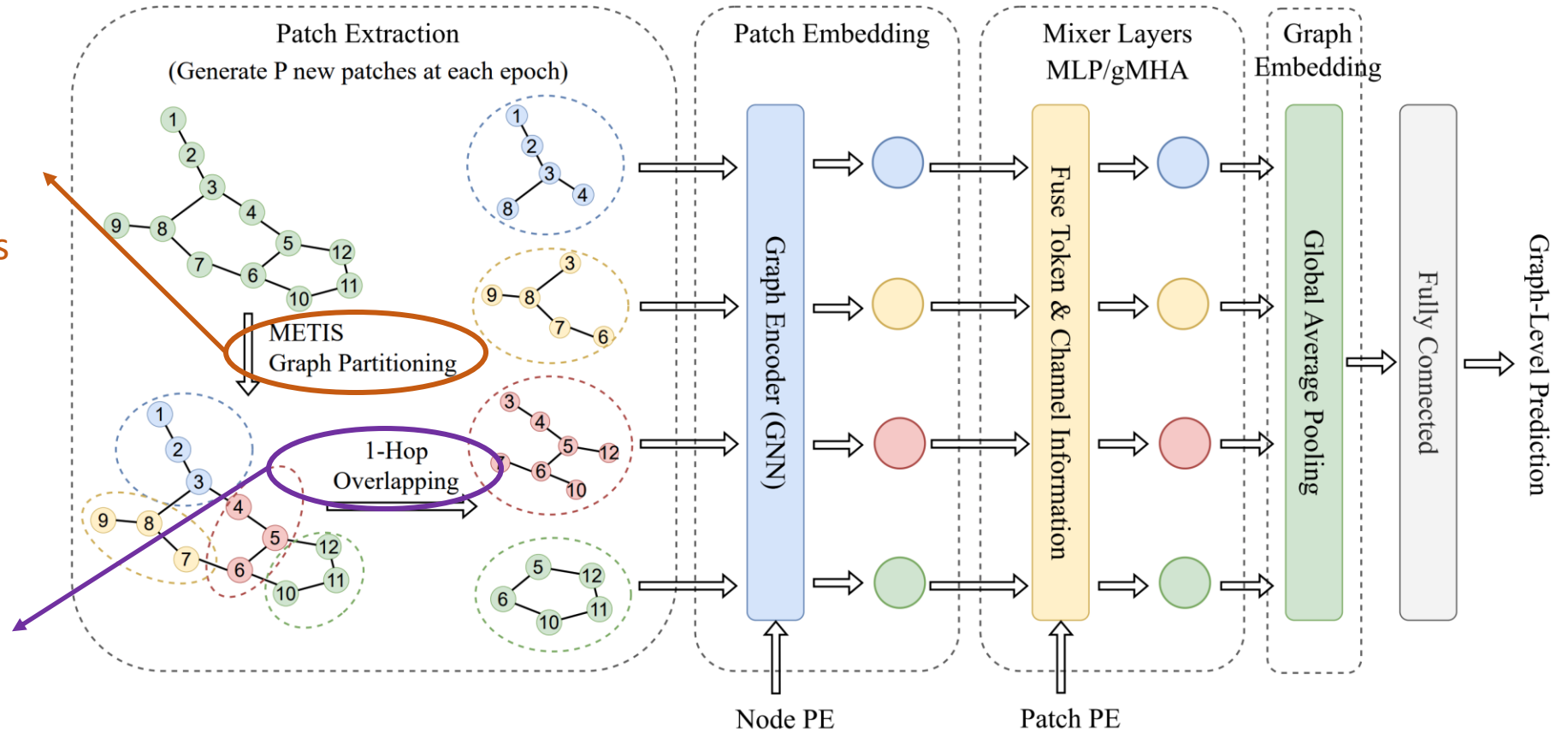
- Use **subgraphs** as tokens (Example: [CoarFormer](#), [MLP-Mixer](#))
 - A natural generalization of ViT to graph domain
 - Significantly **reduces the computational complexity**
 - Enable graph transformers to scale to large graphs



MLP-Mixer Architecture

METIS:

- graph partitioning algorithm
- # intra-cluster links is much higher than inter-cluster links
- Involve **all 1-hop neighbors** to capture important edge information
- e.g., the cutting edges



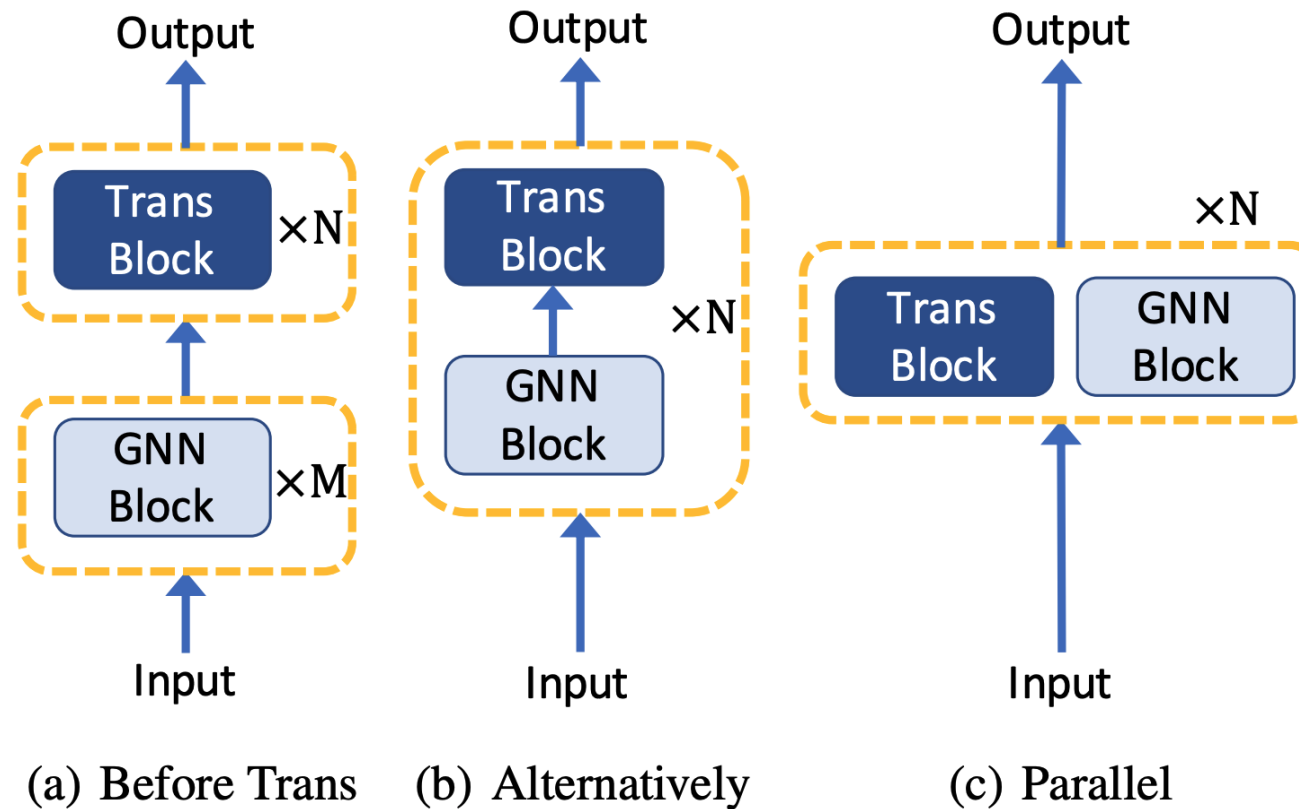
Patch PE counts the number of connecting edges between cluster $\mathcal{V}_i$ and cluster $\mathcal{V}_j$ $A_{ij}^P = |\mathcal{V}_i \cap \mathcal{V}_j|$

Content

- Motivation of Graph Transformers
- Challenges for Building Graph Transformers
- Encodings: Positional & Structural
 - Laplacian Positional Encoding
 - Random Walk Structural Encoding
- Token Construction
- **Forward Propagation**
- Scalability
- Empirical Verification

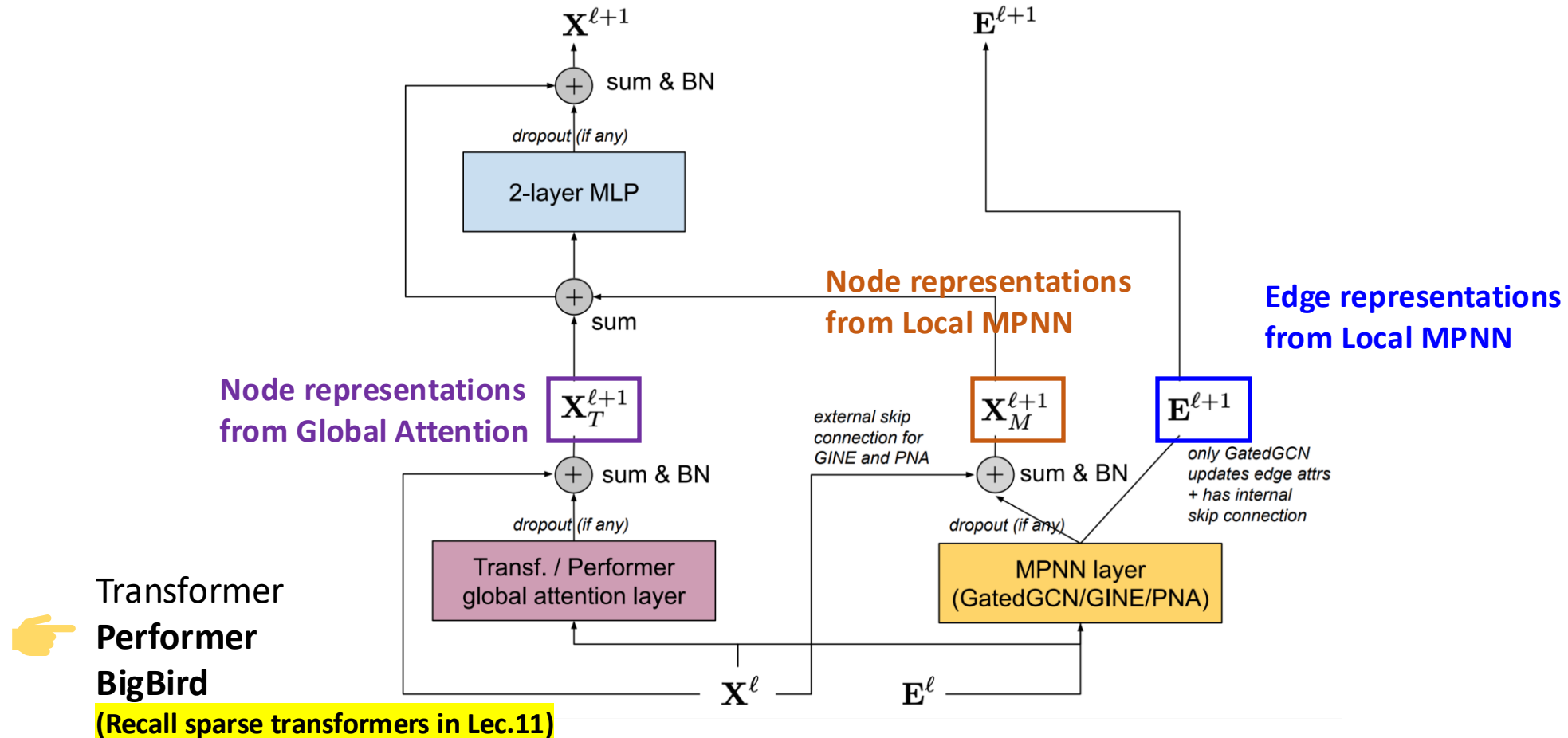
Forward Propagation

- GNNs can be used as **auxiliary modules with transformer** architectures



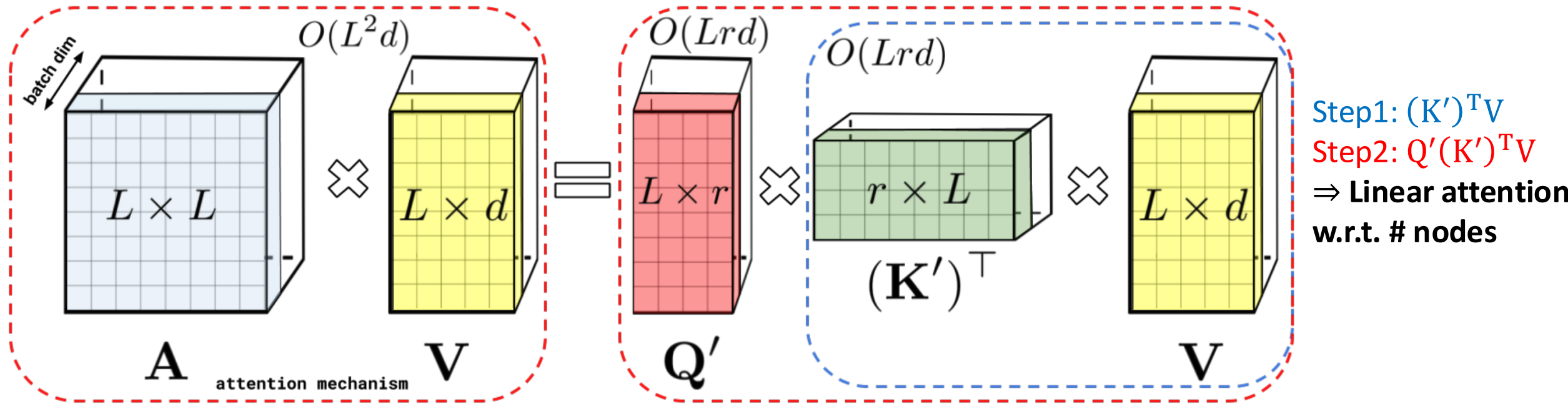
General, Powerful, Scalable (GPS) layers

- GPS Layer combines local MPNN and global attention blocks in parallel



Performer in GPS

- Main idea: **change order of matrix multiplication** to reduce complexity



Why do we not use the original Q and K matrices?

- Note: other linear attention mechanisms (e.g., [BigBird](#)) can also be used to accelerate full attention.

Experimental Results of Graph Transformers

method	#param.	test MAE
GIN [54]	509,549	0.526 ± 0.051
GraphSage [18]	505,341	0.398 ± 0.002
GAT [50]	531,345	0.384 ± 0.007
GCN [26]	505,079	0.367 ± 0.011
GatedGCN-PE [4]	505,011	0.214 ± 0.006
MPNN (sum) [15]	480,805	0.145 ± 0.007
PNA [10]	387,155	0.142 ± 0.010
GT [13]	588,929	0.226 ± 0.014
SAN [28]	508,577	0.139 ± 0.006
Graphormer _{SLIM}	489,321	0.122 ± 0.006

Dataset: ZINC

method	#param.	AUC (%)
GCN-GraphNorm [5, 8]	526K	78.83 ± 1.00
PNA [10]	326K	79.05 ± 1.32
PHC-GNN [29]	111K	79.34 ± 1.16
DeeperGCN-FLAG [30]	532K	79.42 ± 1.20
DGN [2]	114K	79.70 ± 0.97
GIN-vn[54] (fine-tune)	3.3M	77.80 ± 1.82
Graphormer-FLAG	47.0M	80.51 ± 0.53

Dataset: OGB-MolHIV

Graph Transformers (e.g., Graphormer) can outperform message passing GNNs in many tasks

Content

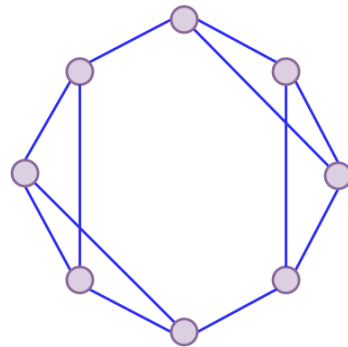
- Motivation of Graph Transformers
- Challenges for Building Graph Transformers
- Encodings: Positional & Structural
 - Laplacian Positional Encoding
 - Random Walk Structural Encoding
- Token Construction
- Forward Propagation
- **Scalability**
- Empirical Verification

Scalability

- Standard (dense) transformers have **quadratic complexity** in the number of tokens, which limits their scalability to extremely long sequences.
- Possible reason: linear attention mechanisms are **designed for natural sequences** (e.g., language, images)
- Question: how to design **specialized attention** mechanisms that are more tailored to learning interactions on **general graph**?

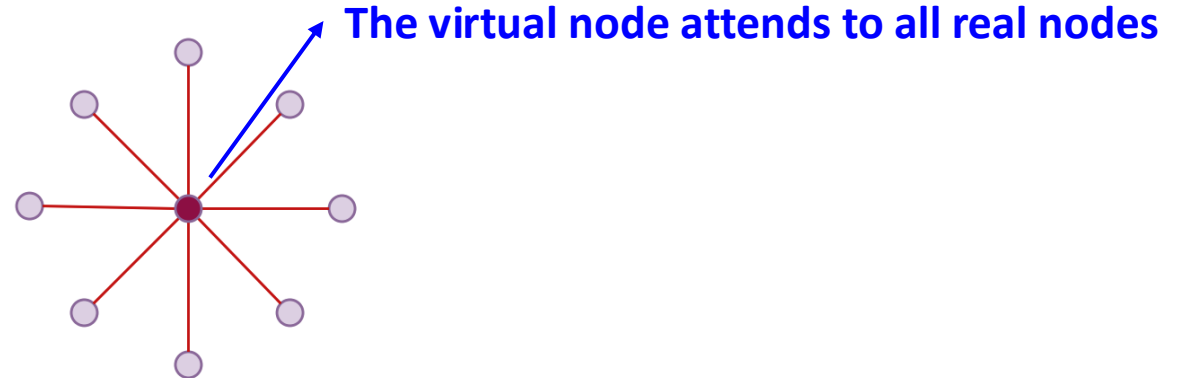
Exphormer: Sparse Transformers for Graphs

- Key point: design a **graph-centric sparse attention** mechanism that makes use of the underlying structure of the input graph



Expander graph
with degree 3

edges: $O(N)$
(will come back)



Global attention
with a virtual node

edges: $O(N)$

Random Expander Graph

d-regular Expander Graph Generation

- Given a node set V , pick $d/2$ permutations $\pi_1, \pi_2, \dots, \pi_{d/2}$ on V , each π_i are chosen independently and uniformly from all possible permutations
- Then, generate edge sets:

$$E = \left\{ \left(i, \pi_j(i) \right), \left(i, \pi_j^{-1}(i) \right) \mid j \in \left\{ 1, 2, \dots, \frac{d}{2} \right\}; i \in \{1, 2, \dots, |V|\} \right\}$$

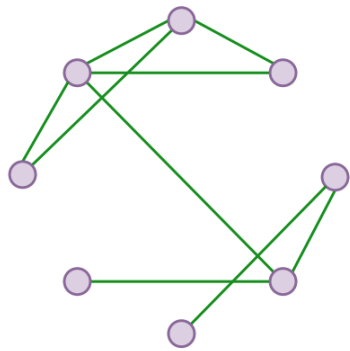
- The number of edges is $d|V|$. The degree of each node is d .

Theorem [1]: For a sparse attention mechanism generated by a d -regular expander on n nodes, **stacking $O(\log n)$ transformer layers** models full attention.

[1] Shirzad, Hamed, et al. "Expformer: Sparse transformers for graphs." *arXiv preprint arXiv:2303.06147* (2023).

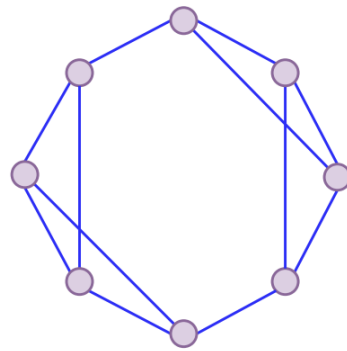
Expander: Long-range Dependencies

- **The expander graph** and **virtual node** allow propagating information between pairs of nodes that are distant in the input graph G without connecting all pairs of nodes.



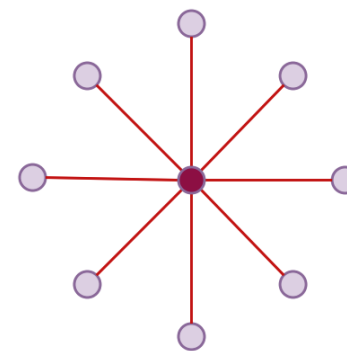
(a)

Local attention
(i.e., real edges of the graph)
edges: $O(E)$



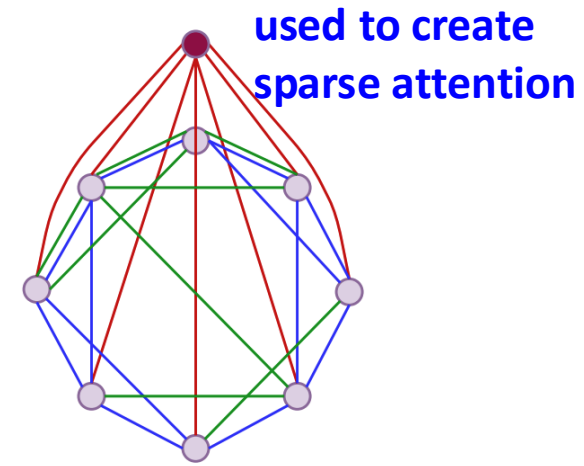
(b)

Expander graph
with degree 3
edges: $O(N)$



(c)

Global attention
with a virtual node
edges: $O(N)$



(d)

Combination of all
attention components
edges: $O(N + E)$

Experimental Results

- The complexity of the combination attention is $O(N + E)$, N, E are numbers of nodes and edges.

Edges in the
original graph
are significant!

The effect of virtual nodes and
expander graphs depends on
the dataset.

Dataset	No Local Edges	No Expander Edges	No Global Nodes	All Components
Cifar10	74.62 ± 0.12	74.53 ± 0.19	74.68 ± 0.19	74.69 ± 0.13
Malnet-Tiny	92.64 ± 0.55	94.02 ± 0.21	92.48 ± 0.33	92.06 ± 0.18
Pattern	86.59 ± 0.03	86.70 ± 0.02	86.14 ± 0.08	86.74 ± 0.02
PascalVOC-SP	0.3708 ± 0.0039	0.3588 ± 0.0013	0.3975 ± 0.0037	0.3682 ± 0.0042
Peptides-Struct	0.2631 ± 0.0007	0.2481 ± 0.0007	0.2655 ± 0.0003	0.2643 ± 0.0008
Computer	90.34 ± 0.45	91.48 ± 0.41	91.59 ± 0.31	91.43 ± 0.53

Content

- Motivation of Graph Transformers
- Challenges for Building Graph Transformers
- Encodings: Positional & Structural
 - Laplacian Positional Encoding
 - Random Walk Structural Encoding
- Token Construction
- Forward Propagation
- Scalability
- **Empirical Verification**

Empirical Verification (Expressiveness)

Model	Easy	Medium		Hard
	EDGES	TRIANGLES-SMALL	TRIANGLES-LARGE	CSL
	2-way Accuracy $\uparrow$	10-way Accuracy $\uparrow$	10-way Accuracy $\uparrow$	10-way Accuracy $\uparrow$
GIN	98.11 ± 1.78	71.53 ± 0.94	33.54 ± 0.30	10.00 ± 0.00
Transformer	55.84 ± 0.32	12.08 ± 0.31	10.01 ± 0.04	10.00 ± 0.00
Transformer (LapPE)	98.00 ± 1.03	78.29 ± 0.25	10.64 ± 2.94	100.00 ± 0.00
Transformer (RWSE)	97.11 ± 1.73	99.40 ± 0.10	54.76 ± 7.24	100.00 ± 0.00
Graphormer	97.67 ± 0.97	99.09 ± 0.31	42.34 ± 6.48	90.00 ± 0.00

EDGES: Predict if an edge connects two nodes in the graph

TRIANGLES: count the number of triangles in the graph
LARGE: train/val graphs are much smaller than test graphs

Circular Skip Links Graphs

- Graph Transformers with structural bias generally perform well on all three tasks with a few exceptions.
- The shortest-path encoding in Graphormer distinguishes 9 out of the 10 classes correctly in CSL dataset
- All graph transformers **generalize poorly to larger triangle dataset** $\Rightarrow$ **still suffer from limited expressiveness**

Empirical Verification (Oversmoothing)

Benchmarking on six graph datasets that especially suffer from the over-smoothing issue of GNNs:

Model (PE/SE type)	ACTOR	CORNELL	TEXAS	WISCONSIN	CHAMELEON	SQUIRREL
Geom-GCN [Pei <i>et al.</i> , 2020]	31.59 $\pm$ 1.15	60.54 $\pm$ 3.67	64.51 $\pm$ 3.66	66.76 $\pm$ 2.72	60.00 $\pm$2.81	38.15 $\pm$ 0.92
GCN (no PE/SE)	33.92 $\pm$ 0.63	53.78 $\pm$ 3.07	65.95 $\pm$ 3.67	66.67 $\pm$ 2.63	43.14 $\pm$ 1.33	30.70 $\pm$ 1.17
GCN (LapPE)	34.30 $\pm$ 1.12	56.22 $\pm$ 2.65	65.95 $\pm$ 3.67	66.47 $\pm$ 1.37	43.53 $\pm$ 1.45	30.80 $\pm$ 1.38
GCN (RWSE)	33.69 $\pm$ 1.07	53.78 $\pm$ 4.09	62.97 $\pm$ 3.21	69.41 $\pm$ 2.66	43.84 $\pm$ 1.68	31.77 $\pm$ 0.65
GCN (DEG)	33.99 $\pm$ 0.91	53.51 $\pm$ 2.65	66.76 $\pm$ 2.72	67.26 $\pm$ 1.53	46.36 $\pm$ 2.07	34.50 $\pm$ 0.87
GPS ^{GCN+Transformer} (LapPE)	37.68 $\pm$ 0.52	66.22 $\pm$ 3.87	75.41 $\pm$ 1.46	74.71 $\pm$ 2.97	48.57 $\pm$ 1.02	35.58 $\pm$ 0.58
GPS ^{GCN+Transformer} (RWSE)	36.95 $\pm$ 0.65	65.14 $\pm$ 5.73	73.51 $\pm$ 2.65	78.04 $\pm$2.88	47.57 $\pm$ 0.90	34.78 $\pm$ 1.21
GPS ^{GCN+Transformer} (DEG)	36.91 $\pm$ 0.56	64.05 $\pm$ 2.43	73.51 $\pm$ 3.59	75.49 $\pm$ 4.23	52.59 $\pm$ 1.81	42.24 $\pm$ 1.09
Transformer (LapPE)	38.43 $\pm$0.87	69.46 $\pm$1.73	77.84 $\pm$1.08	76.08 $\pm$ 1.92	49.69 $\pm$ 1.11	35.77 $\pm$ 0.50
Transformer (RWSE)	38.13 $\pm$0.63	70.81 $\pm$2.02	77.57 $\pm$1.24	80.20 $\pm$2.23	49.45 $\pm$ 1.34	35.35 $\pm$ 0.75
Transformer (DEG)	37.39 $\pm$ 0.50	71.89 $\pm$2.48	77.30 $\pm$1.32	79.80 $\pm$0.90	56.18 $\pm$ 0.83	43.64 $\pm$0.65
Graphormer (DEG only)	36.91 $\pm$ 0.85	68.38 $\pm$ 1.73	76.76 $\pm$ 1.79	77.06 $\pm$ 1.97	54.08 $\pm$ 2.35	43.20 $\pm$0.82
Graphormer (DEG, attn. bias)	36.69 $\pm$ 0.70	68.38 $\pm$ 1.73	76.22 $\pm$ 2.36	77.65 $\pm$ 2.00	53.84 $\pm$ 2.32	43.75 $\pm$0.59

1. Geom-GCN is specialized for over-smoothing issue

2. PE/SE have minimal effect on GCN's performance

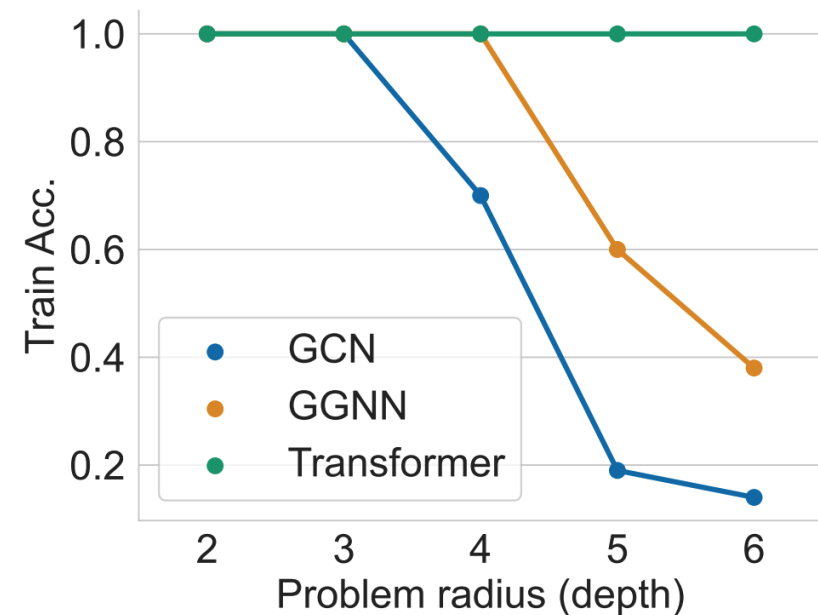
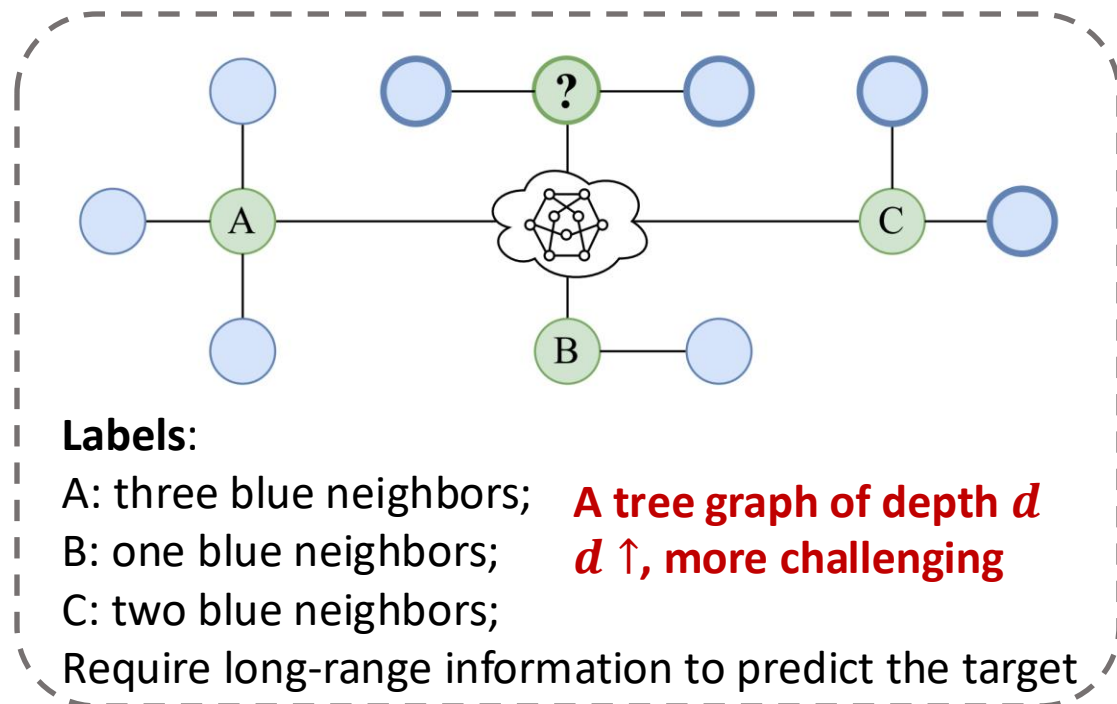
3. Global attention of a transformer empirically facilitates more successful information propagation

Transformer: disabling the local GCN in GPS layers

DEG: using node degree as positional encoding

Empirical Verification (Long-range Dependencies)

- Shallow GNNs often cannot capture long-range dependencies very well
- [Neighbors-Match](#) synthetic dataset:



Graph Transformers have better ability to model **long-range dependencies** and help to **circumvent the over-squashing issue**.

Summary

- Graph Transformer helps to improve the **expressiveness**, alleviate **over-smoothing** and **over-squashing** issues.
- Challenges for graph transformer: positional encoding, structure encoding, scalability.
- Two typical encodings for positions and structures: **LapPE** and **RWE**
- Better PE/SE improves expressiveness.
- Token construction: node-only, node and edge, subgraph (a solution to extremely large-scale graphs)
- GNNs can be used as **auxiliary modules with transformer** architectures.